

Digital Clock

Arjun R Iyengar
ee26btech11202

Arya Sudhirkumar Neel
ee26btech11203

August 2026

Phase 1: Learning the Theory and Connections

- The project was begun by learning the theory about binary operations, how decimal numbers can be converted into binary, and how each output binary digit can be expressed as a function of the inputs. We learnt the Karnaugh Maps(K-Maps) method for generating the functions and using this to increment(or decrement) the digits in the displays of the clock. We learnt the basics of coding in arduino, and the basic functions like pinMode, digitalWrite, millis, digitalRead etc.
- The components for building the clock include: Arduino Uno, IC 7447, Seven Segment Display(Common Anode), Breadboard, Jumper Wires, Resistors, Push Buttons
- In the next step, we learnt the purpose of using the IC 7447:
 1. It translates 4-bit Binary Coded Decimal (BCD) values (0000 to 1001) from the Arduino into the specific combinations of 7 segment lines (a–g) needed to display digits 0–9. Without it, the software code would have to manually calculate and output individual bit patterns for every single display update.
 2. The display receives those 7 segment signals and lights up the corresponding LED lines. At the same instant, the Arduino powers the common anode pin (D4–D9) of the targeted digit to turn it on.
that appears continuous.
- Understanding multiplexing:
 1. Without multiplexing, every single digit display must be wired independently so it can receive its own unique data continuously.
 2. Multiplexing connects all display digits to the exact same 4-wire data bus, cutting down hardware drastically. We then add 1 extra "digit select" control pin per display to switch power ON or OFF to each digit individually.
 3. Persistence of Vision: Each digit is displayed for 1ms, creating a fast alternating effect:

$$t_{\text{digit}} = 0.001s \implies t_{\text{cycle}} = 0.006s$$
$$f_{\text{refresh}} = 1/t_{\text{cycle}} = 166.67\text{hz}$$

As this frequency is much higher than 60hz, our eyes are unable to perceive the individual displays turning ON and OFF sequentially.

- Understanding the connections to be made:

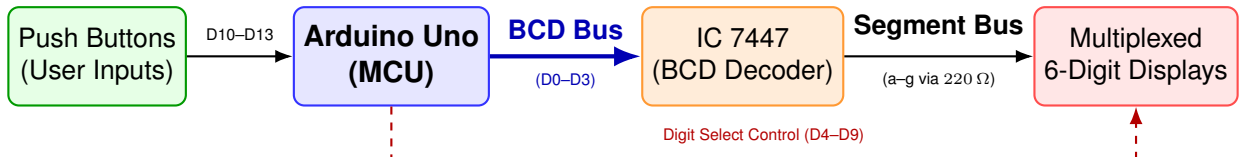


Figure 1: High-Level System Overview Block Diagram

Phase 2: Making the First Set of Connections

The following set of connections were made to get a feel of the process, and to test a single display (display 6)

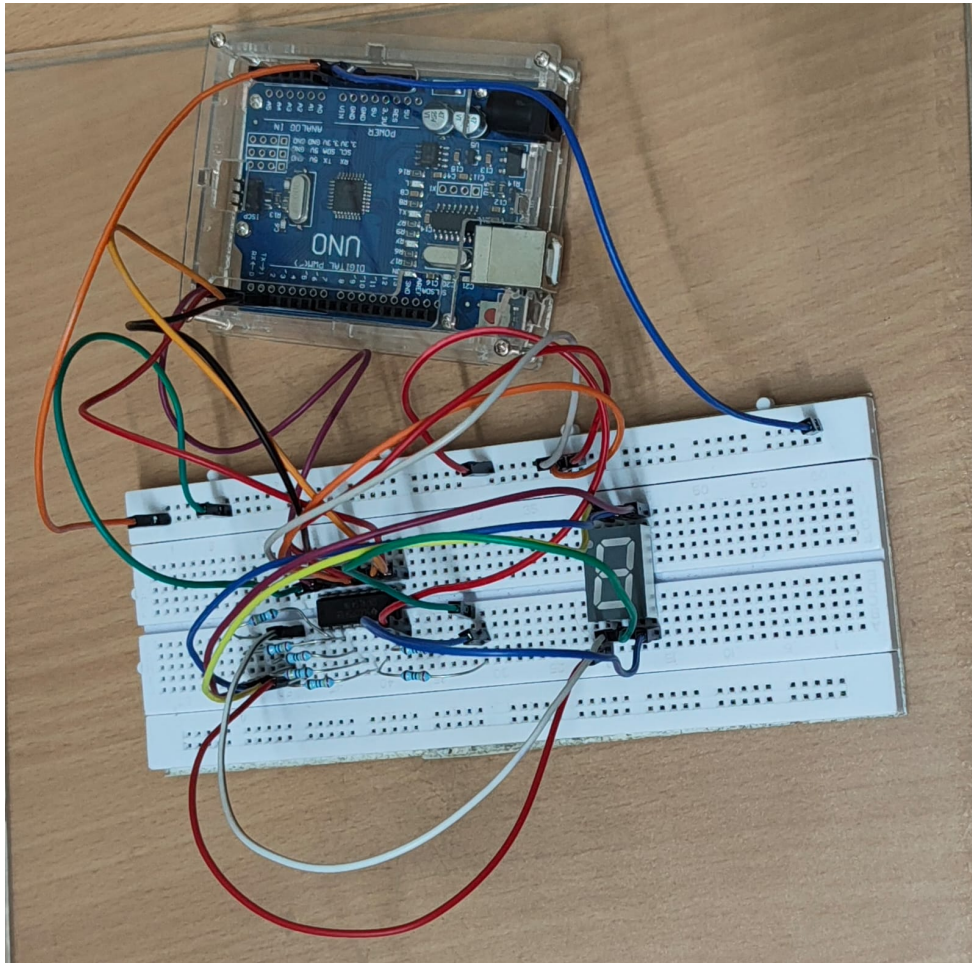


Figure 2: First set of Connections

Phase 3: Coding and debugging for first display

Code

We began writing the code, and got used to the logic of the code, using the `pinMode`, `digitalWrite` functions and setting up the functions to display the digits and increment them.

Fixing the errors

1. The first attempt was to change the display to check if it was faulty, which did not work, which was followed by checking for loose connections and changing wires, which also failed, as all connections were correct.
2. Upon researching, the solution that we found was that the pin 4 of the IC 7447 needed to be connected to the 5V rail on the breadboard. This was needed because even though technically, the clock might appear to work temporarily because the 7447 naturally defaults to a HIGH logic state when the pins are left disconnected. However, it will not work reliably and will introduce erratic display bugs.

Pin 4 ($\overline{\text{BI}}/\text{RBO}$ — Blanking Input): When connected to GND (0V), this pin forcibly shuts OFF all 7 segments, ignoring whatever BCD number is being sent to the chip. Tying it to 5V disables forced blanking, keeping the display outputs active.

3. Another change we made was changing the BCD input pins from D0-D3 to D2-D5, as the D0 and D1 pins are reserved for USB serial communication and uploading code, which caused errors upon uploading our code.

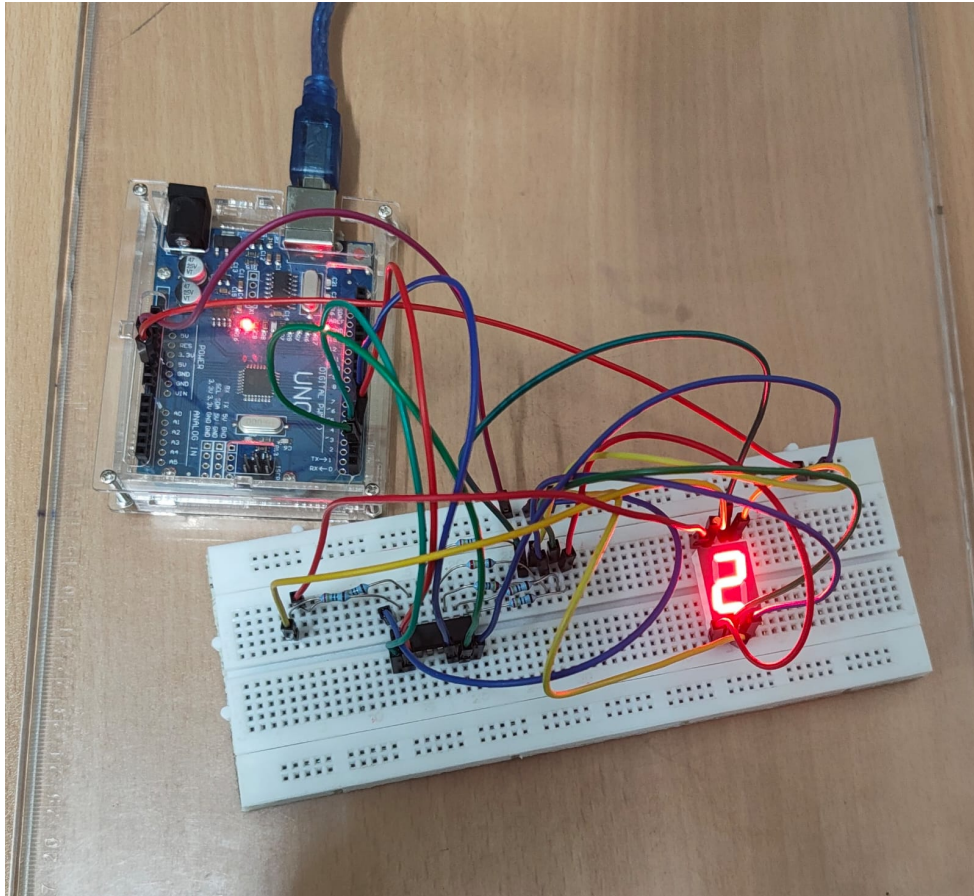


Figure 3: First Display Working

This was followed by testing all the common anode displays and verifying if each one of them counts the seconds from 0-9, so that no error of the display being faulty in the future would be encountered.

Phase 4: Adding all of the Displays

We decided to go step by step and make the connections for each display one by one, so that we can detect errors in each display, instead of finishing the connections to all displays.

1. Upon adding the second, display, we noticed that some segments(e and f) of the display were not lighting up as shown in the figure below:

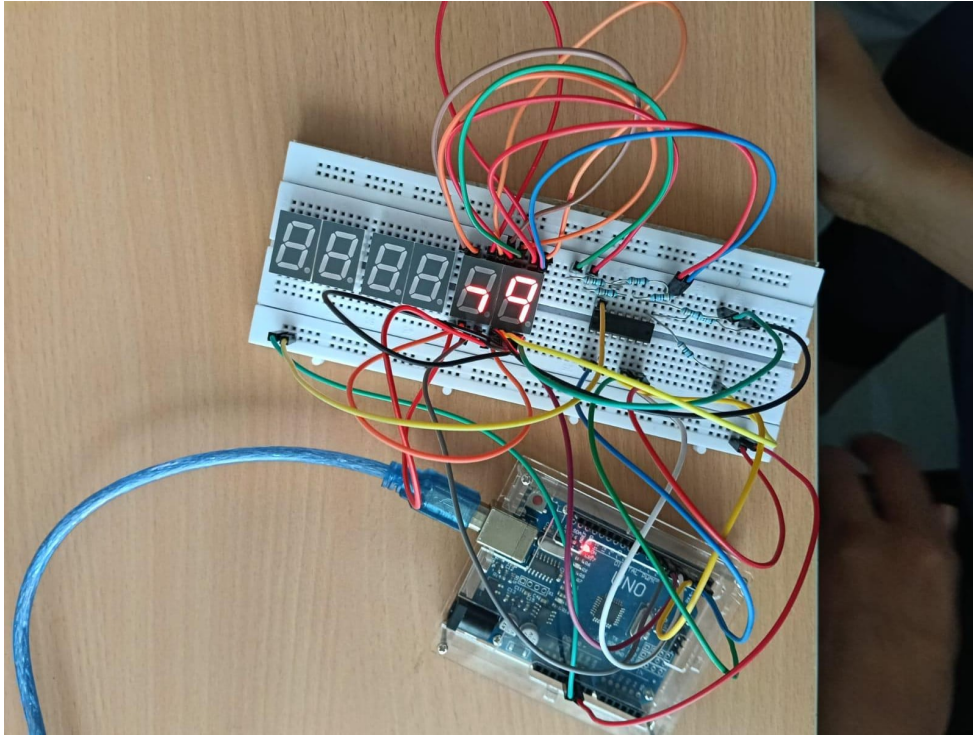


Figure 4: Segment 'f' not lighting up for second display

2. To fix the segment 'f', we fixed the loose connection issues, and for segment 'e', we realised that one column of the breadboard was not supporting correct flow of current, so we moved both the resistor and the jumper wire to another row, fixing the error.
3. No errors were encountered after adding the third display. We encountered another loose connection error upon adding and making the connections for the fourth display, which was fixed.

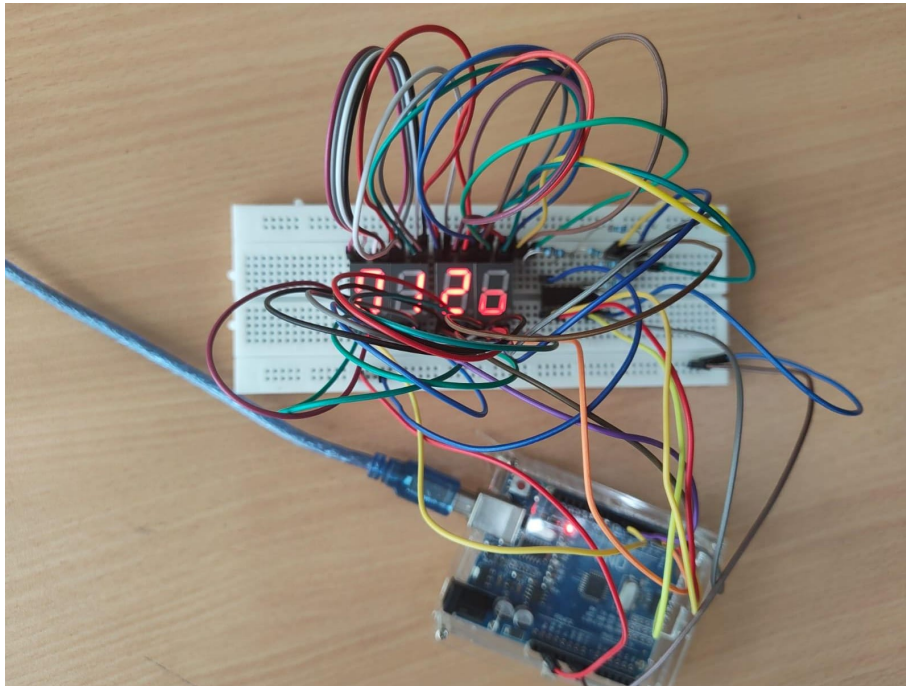


Figure 5: Loose connection error

4. Next, we made the connections for the fifth and sixth segment. This was the progress after adding the fifth display

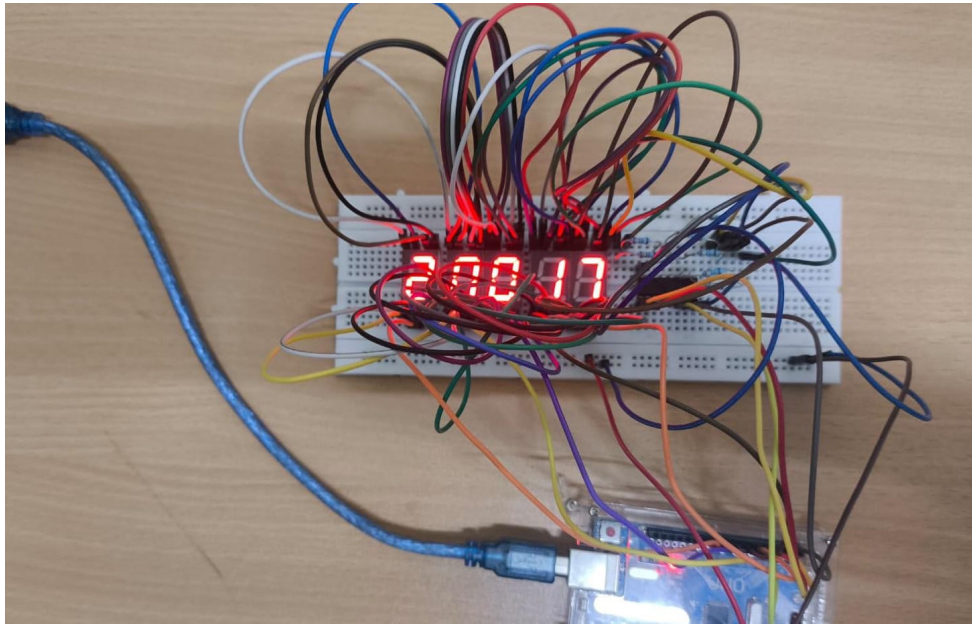


Figure 6: Progress after five Displays

Phase 5: Adding the Buttons

We figured out the mechanism behind which the buttons work, what connections were needed to be made and worked on what needed to be added to the code.

1. The four buttons follow active-HIGH logic, with each button using a $10\text{k}\Omega$ pull-down resistor to guarantee clean 5V (HIGH) signals when pressed and 0 V (LOW) signals when released.
2. One side of every button connects directly to the +5 V power rail, while the opposite side connects to its designated Arduino digital input pin.
3. While idle, the $10\text{k}\Omega$ resistor continuously drains stray current to Ground to hold the pin at 0 V (LOW). Connecting the other side of the switch to +5V creates a low-resistance path to power when pressed, overriding the $10\text{k}\Omega$ resistor and pulling the pin voltage cleanly up to 5 V.
4. The concept of debouncing: Pushing a tactile switch causes its metal contacts to physically bounce for 5–20 ms before establishing a steady connection. To avoid the pin state being read in this window, the millis function is utilised to debounce it.

Upon making the connections to the buttons, we noticed that the next digit selection button was working.

1. To identify the error we tried changing the arduino pins first and discovered the A0 pin to not function with the code.
2. As the increment and 'next' buttons were still not working, upon trial and error, replacing one of the pause button wires, and changing the 'next' button fixed the issue.

However, we encountered an error where the number of hours was going above 23, to say 29

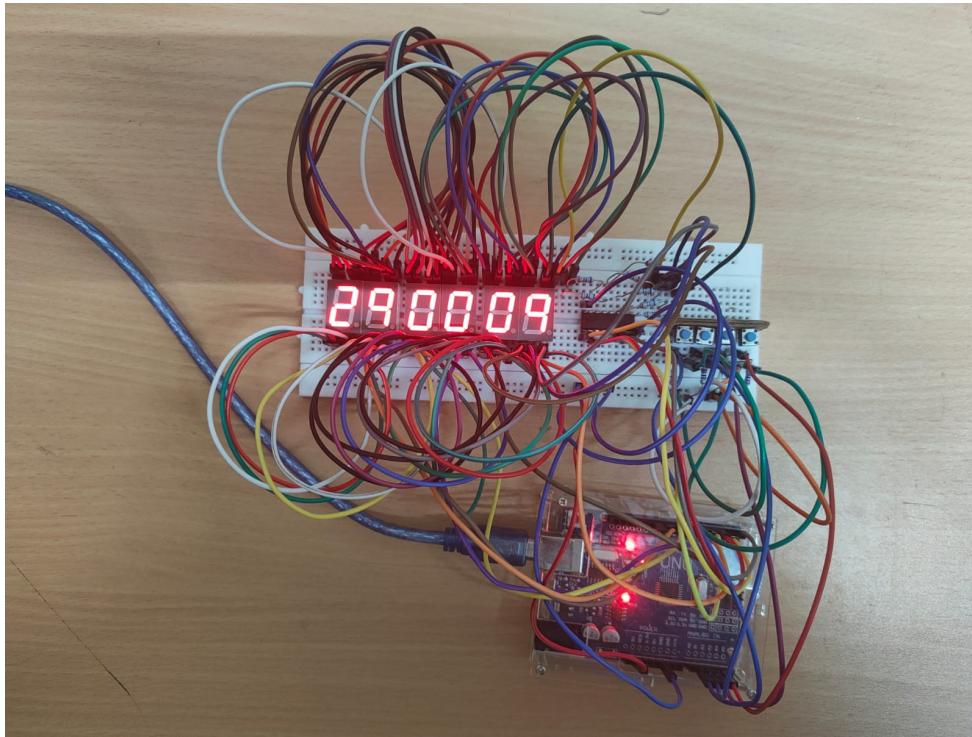


Figure 7: Enter Caption

```

1 case 4:
2     A = !W5;
3     B = (W5 && !X5 && !Z5) || (!W5 && X5);
4     C = (!X5 && Y5) || (!W5 && Y5) || (W5 && X5 && !Y5);
5     D = (!W5 && Z5) || (W5 && X5 && Y5);
6     W5 = A; X5 = B; Y5 = C; Z5 = D;
7     break;
8 // Increment code allowed Hours unit display to go from 0-9 for any value of Hours
   tens display.

```

Incorrect Increment logic

```

1 case 4:
2     if (X6 == 1) {
3         if (Y5 || Z5){
4             W5 = 0; X5 = 0; Y5 = 0; Z5 = 0;
5         } else{
6             A = !W5;
7             B = (W5 && !X5) || (!W5 && X5);
8             W5 = A; X5 = B; Y5 = 0; Z5 = 0;
9         }
10    } else {
11        A = !W5;
12        B = (W5 && !X5 && !Z5) || (!W5 && X5);
13        C = (!X5 && Y5) || (!W5 && Y5) || (W5 && X5 && !Y5);
14        D = (!W5 && Z5) || (W5 && X5 && Y5);
15        W5 = A; X5 = B; Y5 = C; Z5 = D;
16    }
17    break;
18 // Fixed logic for increment code

```

Fixed Increment logic

The decrement logic also had the issue of displaying hours greater than 23, which was fixed by changing the code.

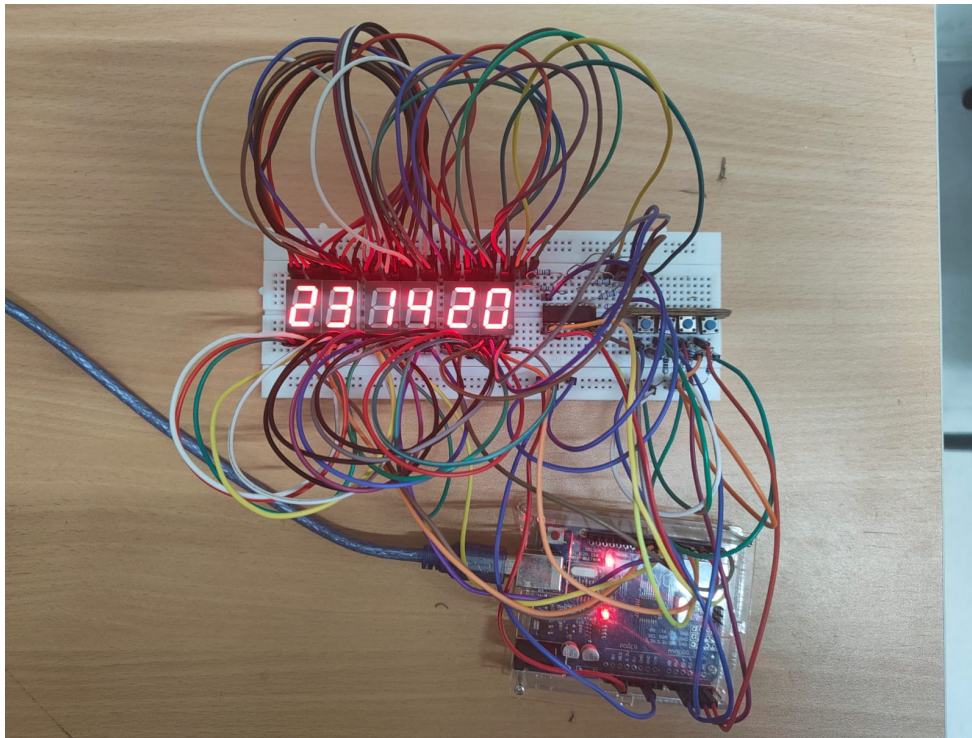


Figure 8: Working Clock

The video of the full working clock :
<https://www.youtube.com/watch?v=0v-uN5F9Gws>